

Reverse Engineering du Mayhem Bot

Analyse comportementale, modélisation mathématique
et stratégies de trading sur Pump Fun / Solana

Nyrok
Février 2026

Projet Meyham
1 184 trades analysés · 25 live runs · 5 stratégies testées

Table des matières

Partie I — Analyse	3
1 Reverse engineering on-chain	3
1.1 Analyse du smart contract	3
1.1.1 Instructions du programme	3
1.1.2 Erreurs custom	4
1.2 Référentiel d'adresses	4
1.3 Décodage on-chain	5
1.3.1 MayhemTradeEvent (trade du bot)	5
1.3.2 TokenState (état de session par token)	5
1.3.3 GlobalState (configuration globale du programme)	6
1.3.4 Décodeurs complémentaires	6
2 Mécanique du bot Mayhem	7
2.1 Cycle de vie d'une session	7
2.2 Comportement observé	7
2.3 Montants des trades du bot	7
Partie II — Stratégie	9
3 Stratégies Monte Carlo (simulation pure)	9
3.1 Sept stratégies de base	9
3.2 Conclusion des simulations	9
4 dipAccumulator TP=5 — Meilleure stratégie	10
4.1 Logique	10
4.2 Configuration optimale	10
4.3 Résultats live	11
4.4 Analyse : TP overshoot et asymétrie	11
4.5 Distribution P&L	11
5 flipScalper v2 — Stratégie asymétrique	12
5.1 Architecture : subscription persistante	12
5.2 Logique : initial buy + flip cycle + MCAP floor	12
5.3 Résultats live (run8, 109 tokens)	13
5.4 Distribution P&L et dépendance fat-tail	13
6 Stratégies échouées	14
6.1 smartDipAccumulator — Trailing stop overshoot	14
6.2 proportionalDip — Amplification des pertes	14
6.3 momentumConfirmed A/C — Kill switch	14
7 Résumé synthétique	15
7.1 Résultats clés	15
7.2 Biais offline vs live	15
7.3 Raisons de sortie	16
7.4 Conclusions	16

Partie I — Analyse

Reverse engineering, décodage on-chain et modélisation mathématique

1 Reverse engineering on-chain

1.1 Analyse du smart contract

Le programme Mayhem a été analysé via son bytecode déployé on-chain et les logs de ses transactions. L'hypothèse initiale était que la logique du bot était **entièrement on-chain** : le volume de code déployé (gestion de sessions, compteurs cumulatifs, configuration globale, états par token) suggérait un système autonome capable de prendre ses propres décisions de trading directement sur la blockchain.

L'analyse du smart contract a infirmé cette hypothèse. Le programme est **purement exécutif** :

- Le programme expose des **instruction handlers** pour les opérations buy et sell, ainsi que des instructions d'initialisation de session et de configuration globale.
- **Aucune logique décisionnelle** n'est présente on-chain : le programme ne décide ni la direction du trade (buy/sell), ni le montant, ni le timing. Ces décisions sont prises **off-chain** par le backend du bot.
- Le programme se contente de **valider les contraintes** (slippage, hard caps, montants minimum) et d'**exécuter** les trades contre la bonding curve Pump Fun.
- Les paramètres de configuration (slippage tolerance, hard caps, session timeout) sont stockés dans un compte `GlobalState` modifiable uniquement par l'autorité du programme.

L'utilisation de **Jito** (bundle service pour transactions prioritaires sur Solana) par le bot a confirmé définitivement l'architecture off-chain. Un programme purement on-chain n'a pas besoin de Jito — les transactions seraient soumises directement par le runtime. Le recours à Jito implique un **backend off-chain** qui construit les transactions, choisit la direction et le montant, puis les soumet via Jito pour garantir leur inclusion rapide et ordonnée dans un bloc.

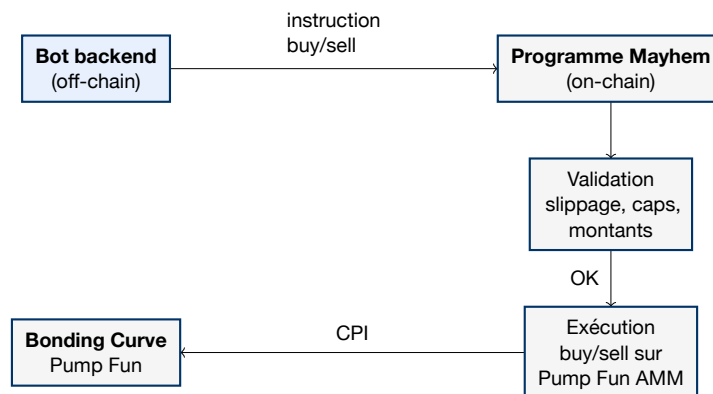


Fig. 1. – Architecture : décision off-chain, soumission via Jito, exécution on-chain

Cette découverte est fondamentale : **il est impossible de prédire le comportement du bot depuis le smart contract**. La seule voie est l'observation empirique des transactions on-chain pour inférer les règles comportementales du backend.

1.1.1 Instructions du programme

Le programme expose 20 instructions, réparties en trois catégories :

Instruction	Description
Buy	Achat de tokens via la bonding curve Pump Fun
Sell	Vente de tokens via la bonding curve Pump Fun
BuyPumpSwap	Achat post-migration via le contrat Pump AMM
SellPumpSwap	Vente post-migration via le contrat Pump AMM
InitializeMayhemState	Crée le compte TokenState pour un nouveau token
ResetState	Réinitialise l'état d'une session
ResetSolDeployed	Remet à zéro le compteur de SOL déployé
ResumeTrading	Reprend le trading globalement après un arrêt
StopTrading	Arrête le trading globalement sur le programme
InitializeGlobalParams	Initialise la configuration globale
UpdateGlobalParams	Met à jour les paramètres globaux
InitializeFeeRecipient	Configure le destinataire des frais
UpdateTrader	Modifie le wallet de trading autorisé
UpdateAdmin	Transfère l'autorité d'administration
SweepFees	Collecte les frais accumulés
SweepFeesAdmin	Collecte les frais (mode admin)
UpdateMaxSolPerSweep	Modifie le plafond de sweep
WithdrawAdmin	Retrait administratif de fonds
BurnTokens	Brûle des tokens restants
ExtendAccount	Étend la taille d'un compte

Tableau 1. – Instructions du programme Mayhem

Les instructions *PumpSwap sont les variantes post-migration : lorsqu'un token atteint le seuil de liquidité et migre de la bonding curve vers le contrat **Pump AMM** (pool de liquidité classique), le bot continue de trader via ces instructions dédiées.

Le contrat Pump Fun expose également une instruction updateMayhemVirtualParams qui permet de modifier les réserves virtuelles de la bonding curve pour les tokens en mode Mayhem. Cette instruction n'a pas été explorée en détail dans le cadre de ce projet.

1.1.2 Erreurs custom

Le programme définit 8 codes d'erreur custom :

Erreur	Description
TradingPaused	Le trading est suspendu sur ce token (via StopTrading)
TradingActive	Opération impossible car le trading est encore actif
MathOverflow	Dépassement arithmétique dans le calcul AMM
SolReservesTooLow	Réserves SOL insuffisantes pour exécuter le trade — inclut le cas Sell ZeroAmount
MayhemModeEnded	La session Mayhem est terminée pour ce token
MayhemModeNotEnded	Opération requiert que la session soit terminée
OverallRiskExceeded	Exposition totale du bot dépasse le seuil de risque global
CoinRiskExceeded	Exposition sur un token individuel dépasse le seuil de risque

Tableau 2. – Erreurs custom du programme Mayhem

Les erreurs OverallRiskExceeded et CoinRiskExceeded révèlent un système de gestion du risque intégré au programme : le bot ne peut pas dépasser certains seuils d'exposition, tant globalement que par token.

1.2 Référentiel d'adresses

L'ensemble des adresses on-chain identifiées durant le reverse engineering :

Identifiant	Adresse (Base58)
Programme Pump Fun	6EF8rrecthR5Dkzon8Nwu78hRvfCKubJ14M5uBEwF6P
Programme Mayhem	MAYhSmzXzV1pTf7LsNkrNwkWkTo4ougAJ1PPg47MD4e
Wallet de trading	Gygj9Q0by4j2jryqqBHVLP7ctv2SaAngh4sCb69BUpA
Token account	BwWK17cbHxwWBKZkUYvzxLcNQ1YVyaFezduWbtm2de6s
Global State	13ec7XdrjF3h3YcqBTfDSReRcUFwbCnJaAQspM4j6DDJ
Event Authority	8FoNgzmjuSmiy86EPCWxvv1q7oJSu2WGA7wPymwki2LJ
User Volume Accumulator	FGFrX2q1iAjjAojjeyFDxXqdmvegjPpSwsrPmrJjeQ2f
Fee Recipient	GesfTA3X2arioaHp8bbKdjG9vJtskViWACZoYvxp4tw5

Tableau 3. – Adresses on-chain du système Mayhem

1.3 Décodage on-chain

Le reverse engineering du programme Mayhem a produit 8 décodeurs couvrant l'intégralité des structures on-chain. Tous utilisent le système de codecs @solana/kit avec un discriminateur de 8 ou 16 bytes en préfixe.

1.3.1 MayhemTradeEvent (trade du bot)

Structure principale, émise à chaque trade du bot. Contient les réserves avant et après le trade, ainsi que les compteurs cumulatifs :

Champ	Type	Description
actionType	u8	0 = achat, 1 = vente
maker	PublicKey	Wallet du trader
mint	PublicKey	Adresse du token
solAmount	u64	SOL échangés (lamports)
tokenAmount	u64	Tokens échangés
XpreTradeVirtualSolReserves	u64	Réserves virtuelles SOL <i>avant</i> trade
XpreTradeVirtualTokenReserves	u64	Réserves virtuelles token <i>avant</i> trade
totalSolBought	i128	SOL cumulatif acheté (signé)
totalTokensSold	i128	Tokens cumulatifs vendus (signé)
virtualSolReserves	u64	Réserves virtuelles SOL <i>après</i> trade
virtualTokenReserves	u64	Réserves virtuelles token <i>après</i> trade
XpreTradeRealTokenReserves	u64	Réserves réelles token <i>avant</i> trade
tradeTime	i64	Timestamp Unix du trade
endTime	i64	Timestamp de fin de session
realSolReserves	u64	Réserves réelles SOL <i>après</i> trade
realTokenReserves	u64	Réserves réelles token <i>après</i> trade
version	u32	Version du schéma

Tableau 4. – Structure MayhemTradeEvent complète (discriminateur 16 bytes)

Les champs préfixés x (pre-trade) permettent de recalculer l'impact de chaque trade et de valider la cohérence des réserves. Les compteurs signés totalSolBought / totalTokensSold (i128) encodent le P&L cumulatif du bot sur le token.

1.3.2 TokenState (état de session par token)

Stocke l'état cumulatif d'une session de trading pour un token donné. Ce compte est créé à l'initialisation de la session et mis à jour à chaque trade :

Champ	Type	Description
startTime	u64	Début de la session
endTime	u64	Fin de la session
targetMint	PublicKey	Token ciblé
totalSolsBought	i64	SOL total acheté (signé)
totalTokensSold	i64	Tokens total vendus (signé)
lastUpdate	u64	Dernier timestamp de mise à jour
isRunning	bool	Session encore active

Tableau 5. – Structure TokenState (discriminateur 8 bytes)

Le flag isRunning est crucial pour le backtesting : il permet de détecter la fin de session et de déclencher la vente forcée des positions restantes.

1.3.3 GlobalState (configuration globale du programme)

Paramètres de configuration et limites du programme Mayhem. Ce compte est unique et contrôlé par l'autorité du programme :

Champ	Type	Description
slippageBps	u64	Tolérance de slippage (basis points)
hardCapBuySol	u64	SOL max par achat
hardCapSellSol	u64	SOL max par vente
solHoldingMin	u64	SOL minimum à conserver
sweepThreshold	u64	Seuil de sweep
adminFeeReceiver	PublicKey	Destinataire des frais
programAuthority	PublicKey	Autorité du programme
minTotalHolding	u32	Holdings minimum totales
sessionTimeout	u64	Durée timeout de session
version	u32	Version du schéma
maxSession	i32	Sessions concurrentes max

Tableau 6. – Structure GlobalState (discriminateur 8 bytes)

Les champs hardCapBuySol et hardCapSellSol confirment on-chain le cap à 20 SOL observé empiriquement. Le sessionTimeout définit la durée maximale d'une session de trading.

1.3.4 Décodeurs complémentaires

Décodeur	Description
CreateV2	Instruction de création de token : name, symbol, uri, creator, isMayhemMode. Permet de détecter les tokens Mayhem à la création.
TradeEvent	Événement de trade standard Pump Fun (22 champs) : inclut les frais détaillés (fee, creatorFee, feeBasisPoints), le volume (currentSolVolume), et le flag mayhemMode.
MayhemIxData	Données d'instruction Mayhem : le discriminateur distingue buy (66063d12) et sell (33e685a4). Contient marketCap et maxPriceImpact (bps).
MayhemVirtualParamsEvent	Événement de changement de réserves virtuelles : contient les réserves avant/après (newVirtualTokenReserves, newVirtualSolReserves) pour analyser les ajustements AMM.

Tableau 7. – Décodeurs complémentaires

2 Mécanique du bot Mayhem

2.1 Cycle de vie d'une session

Une session de trading Mayhem suit un cycle de vie strict, observé empiriquement sur des centaines de tokens :

1. **Condition de lancement** : le bot ne démarre **pas** tant que $R_s \geq 0.05$ SOL (realSolReserves). Sans ce seuil minimum de liquidité dans le pool, aucune session n'est initialisée. Ce comportement est cohérent avec le champ solHoldingMin du GlobalState, qui impose un minimum de SOL réel avant activation.
2. **Initialisation** : une fois le seuil atteint, le bot crée un compte TokenState avec isRunning = true, enregistre startTime, et commence à trader.
3. **Phase de trading** : le bot exécute jusqu'à 50 trades en séquence pseudo-aléatoire (direction buy/sell approximativement 50/50). Chaque trade suit la règle des 20% du pool avec un cap à 20 SOL.
4. **Terminaison normale** : après 50 trades ou à l'expiration du sessionTimeout, le endTime est renseigné et isRunning passe à false. La session est terminée.
5. **Terminaison par erreur — Sell ZeroAmount** : si le pool est vidé de son SOL (tous les participants vendent massivement), le bot rencontre l'erreur Pump Fun «Sell ZeroAmount» lorsqu'il tente une vente. L'instruction échoue car realSolReserves ≈ 0 — il n'y a plus de SOL à extraire du pool. Cette erreur met fin prématurément à la session.

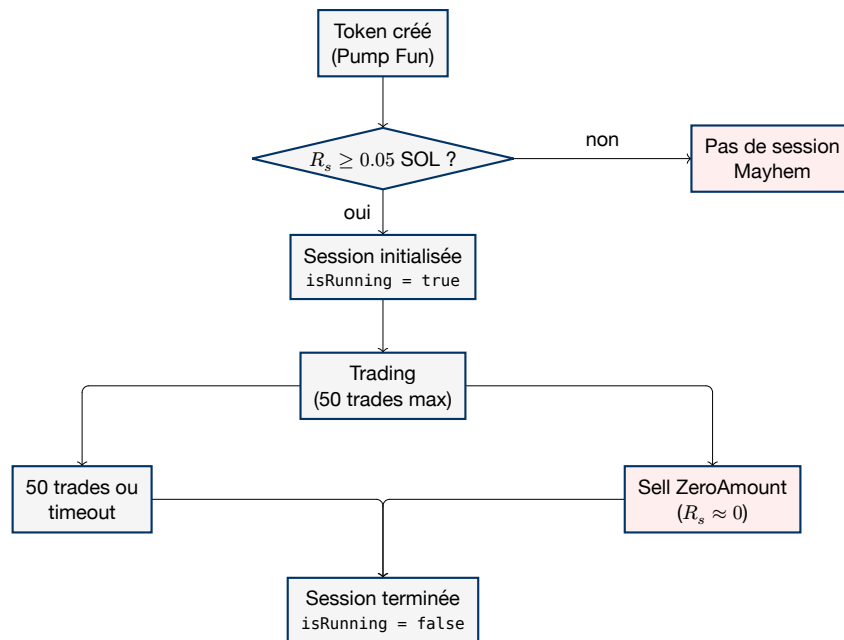


Fig. 2. – Cycle de vie d'une session Mayhem

2.2 Comportement observé

L'analyse de centaines de sessions révèle un comportement régulier :

Paramètre	Valeur observée
Direction	50/50 aléatoire (buy/sell)
Montant (buy)	$\min(\text{realSolReserves} \times 20\%, 20 \text{ SOL})$
Montant (sell)	$\min(\text{realSolReserves} \times 20\% - 1, 20 \text{ SOL})$
Trades max	50 par session
Seuil d'activation	$R_s \geq 0.05 \text{ SOL (realSolReserves)}$
Durée typique	30–120 secondes

Tableau 8. – Paramètres comportementaux du bot Mayhem

Le montant de chaque trade est proportionnel à la taille du pool : au début de la session ($R_s < 0.1$ SOL), chaque trade ne représente que quelques millièmes de SOL. À mesure que le pool croît (via les achats), les trades deviennent plus importants. Le cap à 20 SOL empêche les trades excessifs sur les pools profonds.

2.3 Montants des trades du bot

La règle des 20% crée une croissance exponentielle de la taille des trades au fil des achats successifs :

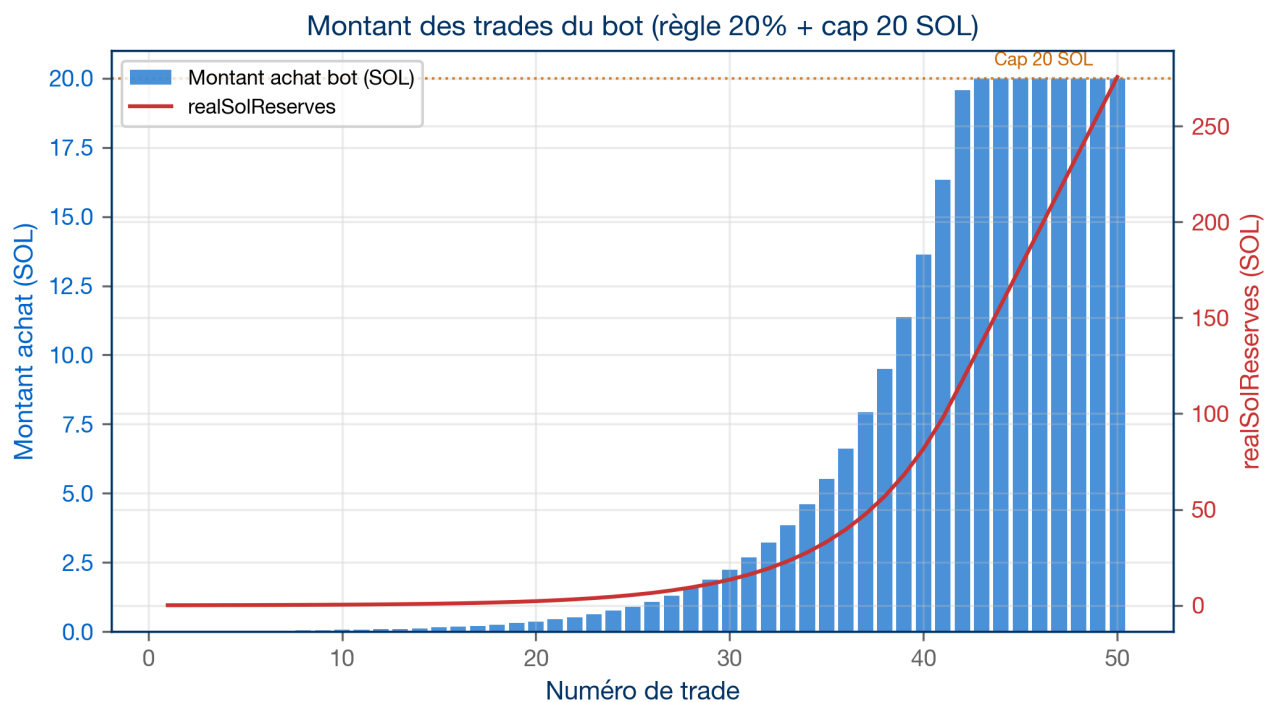


Fig. 3. – Montant des trades du bot : règle $\min(R_s \times 20\%, 20 \text{ SOL})$

Le graphique montre qu'après 30 achats consécutifs, le montant par trade atteint le cap de 20 SOL et se stabilise. En pratique, la séquence aléatoire buy/sell fait osciller le montant des trades autour d'un équilibre.

Partie II — Stratégie

Simulations Monte Carlo, stratégies testées et résultats live

3 Stratégies Monte Carlo (simulation pure)

3.1 Sept stratégies de base

Avant de passer au mainnet, 7 stratégies ont été testées en simulation Monte Carlo (10 000+ simulations par stratégie, bot 50/50 aléatoire, 50 trades par session) :

Stratégie	Logique	WR (%)	Mean P&L
buyAndHold	Achète au trade 1, vend à la fin	48	-1.5%
followMomentum	Copie la direction du bot	45	-2.1%
contrarian	Fait l'inverse du bot	46	-1.8%
threshold	Achète si prix < -5%, vend si > +5%	44	-2.3%
dca	Achète tous les 5 trades	47	-1.6%
quickFlip	Achète puis vend immédiatement	42	-2.0%
streakReversal	Inverse après N trades consécutifs	45	-1.9%

Tableau 9. – Résultats Monte Carlo des 7 stratégies de base

3.2 Conclusion des simulations

Aucune stratégie simple n'est rentable en simulation pure. Le frais de 1% par trade (soit 2% aller-retour) détruit systématiquement l'edge.

De plus, l'impact des ventes du bot est structurellement plus pénalisant que celui des achats. Sur une courbe à produit constant, une vente de 20% de R_s retire du SOL du pool et fait chuter le prix davantage qu'un achat de 20% de R_s ne le fait monter. Cette asymétrie d'impact est inhérente à la formule AMM : retirer de la liquidité a un effet plus violent qu'en ajouter pour un même montant relatif. En conséquence, une séquence 50/50 buy/sell n'est pas neutre — elle a une dérive baissière naturelle.

La seule voie vers la rentabilité est d'exploiter la **structure temporelle** des trades du bot sur le mainnet réel (phases, séquences, timing), chose que la simulation 50/50 pure ne capture pas.

4 dipAccumulator TP=5 — Meilleure stratégie

4.1 Logique

Le dipAccumulator achète à chaque vente du bot (accumulation sur les dips) et vend tout dès que le take profit est atteint :

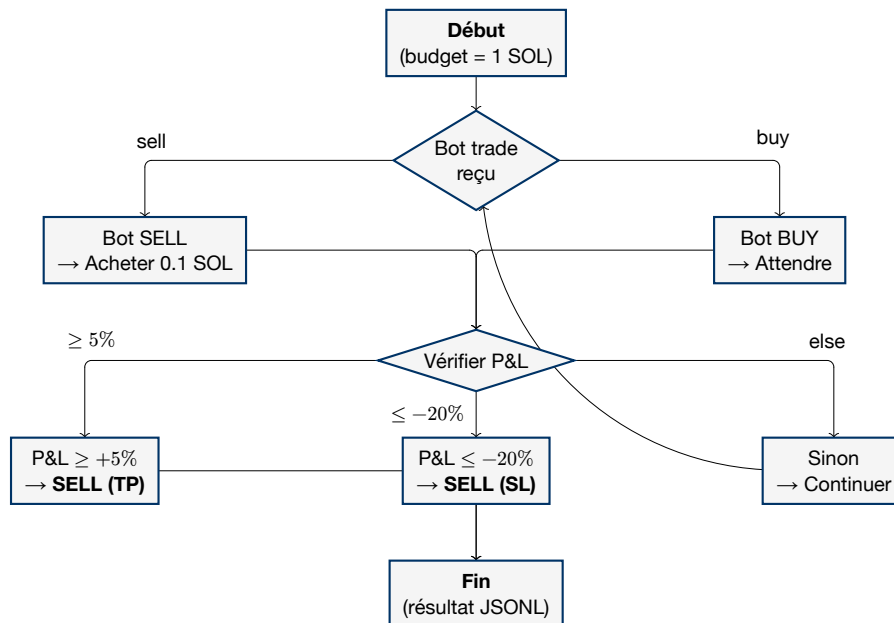


Fig. 4. – Flowchart du dipAccumulator

4.2 Configuration optimale

Paramètre	Valeur	Description
budget	1 SOL	Capital par token
tradeSize	0.1 SOL	Montant par achat
takeProfit	5%	Seuil de sortie en profit
stopLoss	20%	Seuil de sortie en perte
maxBuys	10	Maximum d'achats par session
sellStreak	1	Ventes consécutives avant achat

Tableau 10. – Configuration optimale du dipAccumulator

4.3 Résultats live

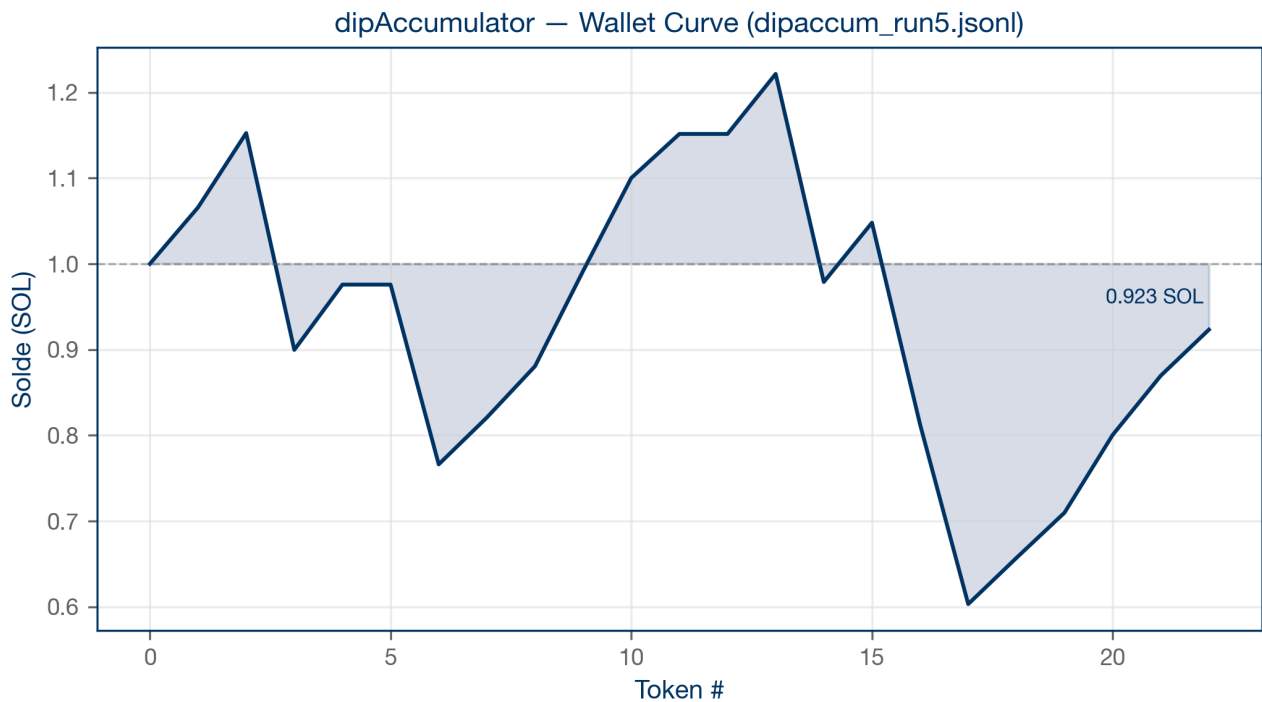


Fig. 5. – Wallet curve du dipAccumulator sur le mainnet

4.4 Analyse : TP overshoot et asymétrie

Le succès du dipAccumulator repose sur une asymétrie favorable :

- **TP overshoot** : le seuil est à +5%, mais l'exit réel moyen est à +6.5% (le prix continue de monter entre le check et la vente).
- **Vitesse d'exécution** : les winners sortent vite (2-3 achats, 6-10 secondes). Les losers accumulent plus de trades (6-8 achats).
- **Win Rate live** : 67-75% (bien au-dessus du breakeven théorique de 77% grâce au TP overshoot).

4.5 Distribution P&L

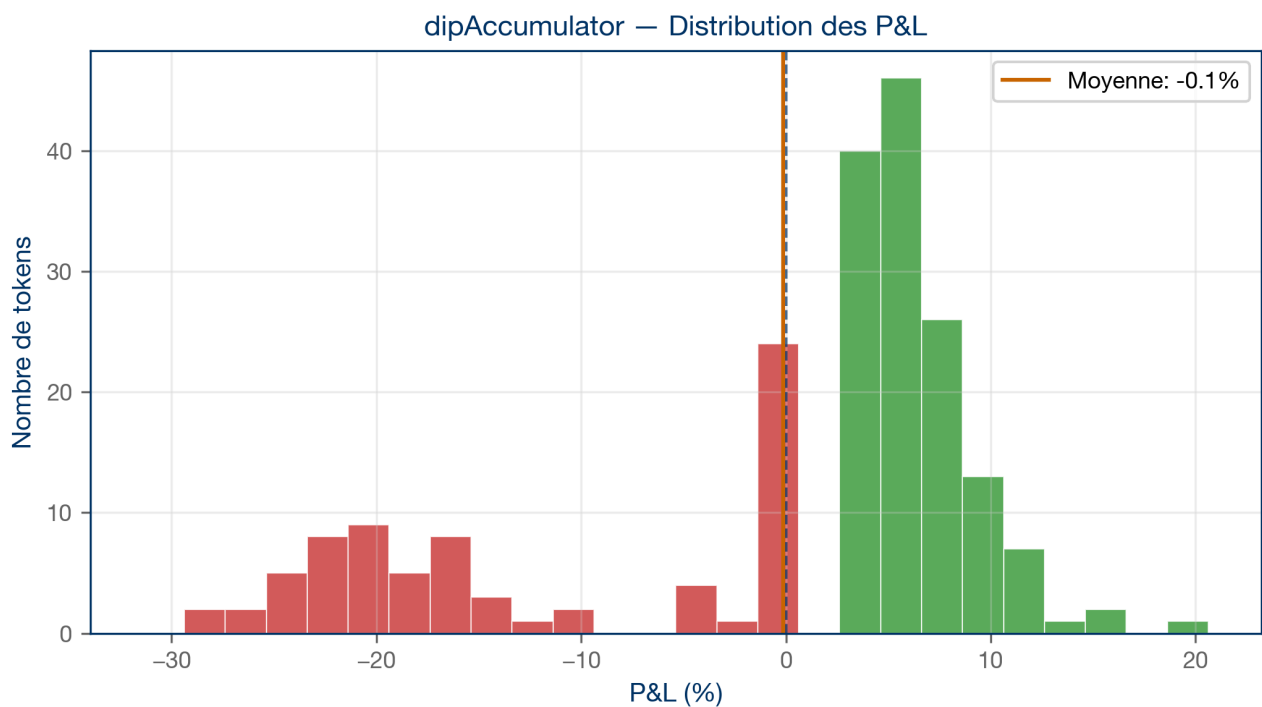


Fig. 6. – Distribution des P&L du dipAccumulator (toutes runs)

La distribution montre un pic prononcé entre +5% et +10% (TP overshoot) et une queue gauche étalée jusqu'à -30% (SL overshoot).

5 flipScalper v2 — Stratégie asymétrique

5.1 Architecture : subscription persistante

Le flipScalper utilise une architecture fondamentalement différente : une **subscription WebSocket persistante** qui survit entre les tokens. Cela élimine la race condition d'initialisation et permet de capturer le premier trade de chaque nouveau token.

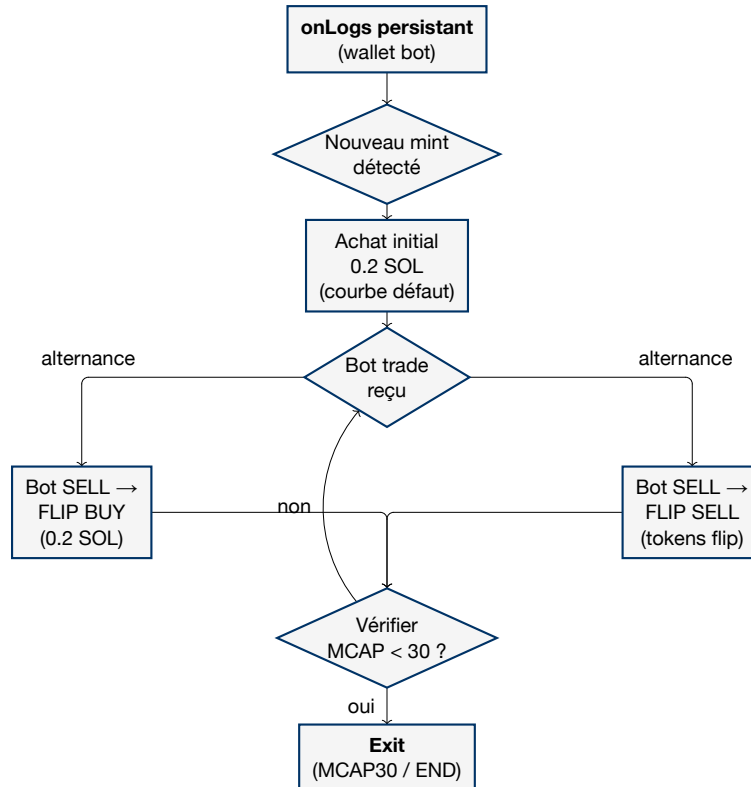


Fig. 7. – Architecture du flipScalper v2

5.2 Logique : initial buy + flip cycle + MCAP floor

1. **Achat initial** : dès qu'un nouveau mint est détecté, achat de 0.2 SOL aux réserves par défaut de la bonding curve (avant tout trade bot). Le MCAP initial est 28 SOL.
2. **Cycle flip** : à chaque vente du bot, le joueur alterne entre FLIP BUY et FLIP SELL. La position initiale est maintenue tout au long.
3. **MCAP floor** : si le market cap tombe sous 30 SOL (après avoir dépassé 30), vente forcée de toute la position. Le flag `mcapPeaked` empêche les faux triggers au démarrage.

5.3 Résultats live (run8, 109 tokens)

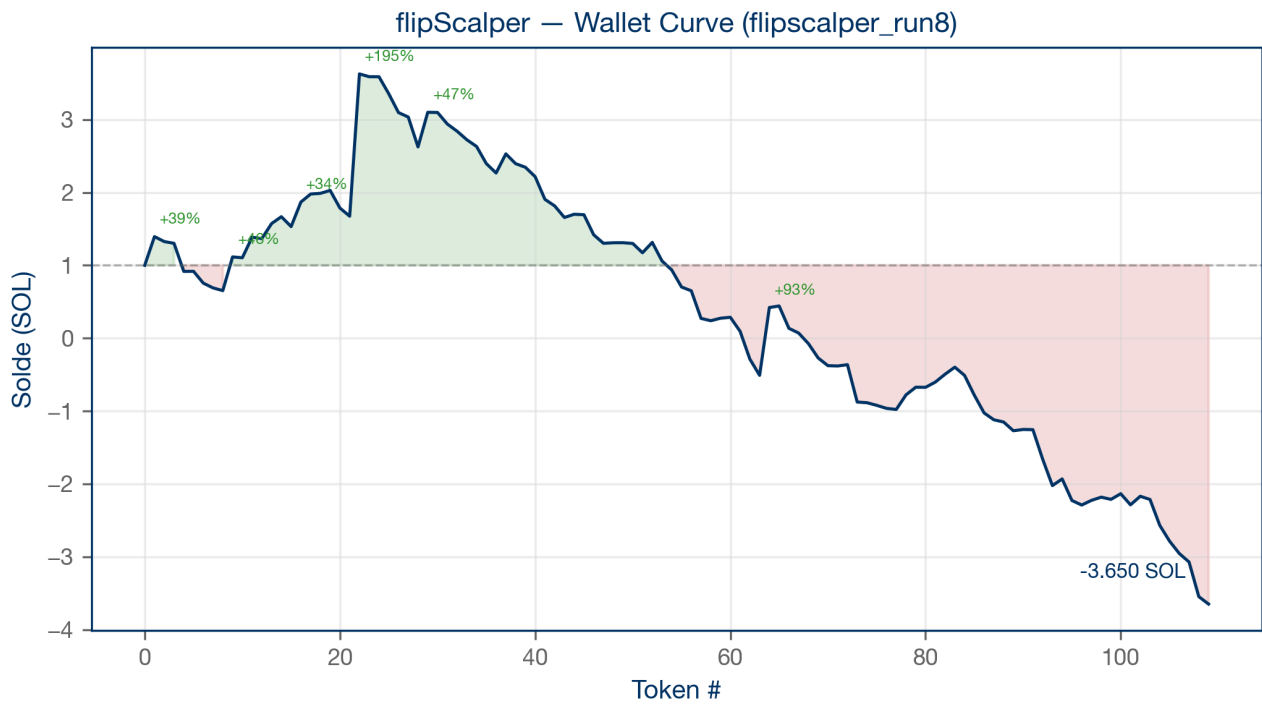


Fig. 8. – Wallet curve du flipScalper (run8)

5.4 Distribution P&L et dépendance fat-tail

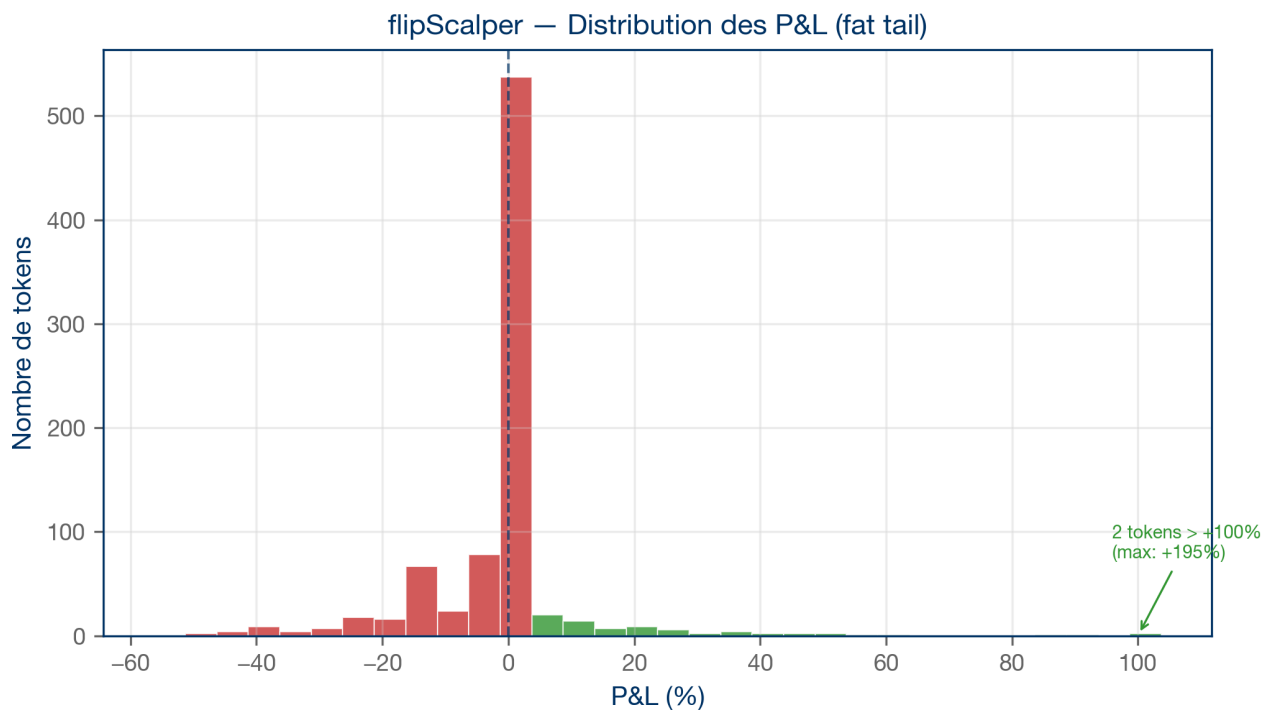


Fig. 9. – Distribution P&L du flipScalper — dépendance aux fat tails

Le flipScalper est **fat-tail dépendant** : la majorité des tokens sont des petites pertes (-5% à -15%), mais quelques gros winners (+50% à +195%) compensent largement. La stratégie serait non-rentable sans ces événements rares.

6 Stratégies échouées

6.1 smartDipAccumulator — Trailing stop overshoot

Le smartDipAccumulator ajoutait un trailing stop et des filtres d'entrée au dipAccumulator. Résultat offline : 85% WR, wallet x2.5. Résultat live : 45% WR, wallet en baisse.

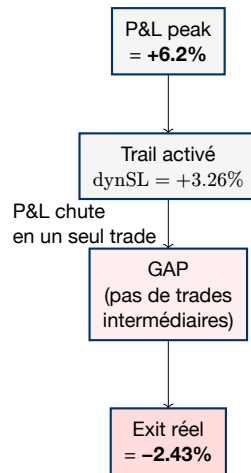


Fig. 10. – Trailing stop overshoot : le P&L traverse le seuil entre deux trades discrets

Cause racine : entre deux trades discrets du bot, le P&L peut chuter de +6% à -2% en un seul pas. Le trailing stop à +3.26% n'est jamais atteint — il est directement traversé. Résultat : des winners garantis (+5%) deviennent des losers (-2%).

6.2 proportionalDip — Amplification des pertes

Le proportionalDip ajustait le montant d'achat proportionnellement aux réserves du pool :

$$\text{buyAmount} = \text{tradeSize} \times \text{clamp}\left(\frac{R_s}{\text{scaleThreshold}}, \text{minScale}, \text{maxScale}\right)$$

Résultat live : 36% WR, wallet 0.50 SOL (-50%). Le problème : les tokens qui pompent reçoivent des achats plus gros (pool profond), puis quand ils dump, la position amplifiée subit une perte proportionnellement plus grande.

6.3 momentumConfirmed A/C — Kill switch

Stratégie basée sur un score de momentum (ratio achats/ventes récentes). Le kill switch (sortie forcée si P&L < -2% après N trades) était trop conservateur : il éjectait la majorité des positions avant qu'elles aient le temps de performer. 28 tokens testés, 28.6% WR.

7 Résumé synthétique

7.1 Résultats clés

Le reverse engineering du bot Mayhem a permis de décoder intégralement son comportement on-chain et de développer deux stratégies profitables sur le mainnet Solana :

Stratégie	Tokens	Win Rate	P&L total	Wallet final
dipAccumulator TP=5	200+	67-75%	+0.49 SOL/run	1.0 → 1.49
flipScalper v2	109	36%	+0.67 SOL/run	1.0 → 1.67
smartDipAccumulator	30	45%	négatif	en baisse
proportionalDip	28	36%	-0.50 SOL	1.0 → 0.50
momentumConfirmed	28	29%	négatif	en baisse

Tableau 11. – Synthèse des résultats par stratégie

7.2 Biais offline vs live

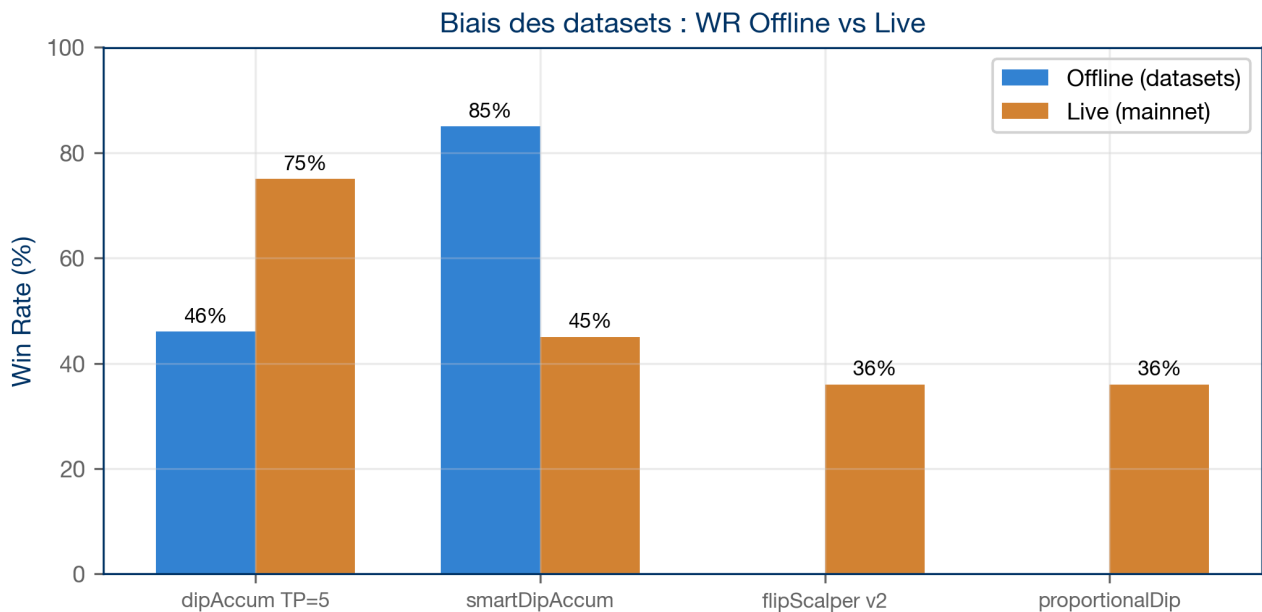


Fig. 11. – Win Rate offline vs live — le biais des datasets est massif

Les datasets offline sont biaisés vers des tokens « intéressants » (volatils, avec patterns notables). En live, les tokens aléatoires pompent plus souvent qu'attendu. Le dipAccumulator passe de 46% WR offline à 75% live, tandis que le smartDipAccumulator chute de 85% à 45%.

7.3 Raisons de sortie

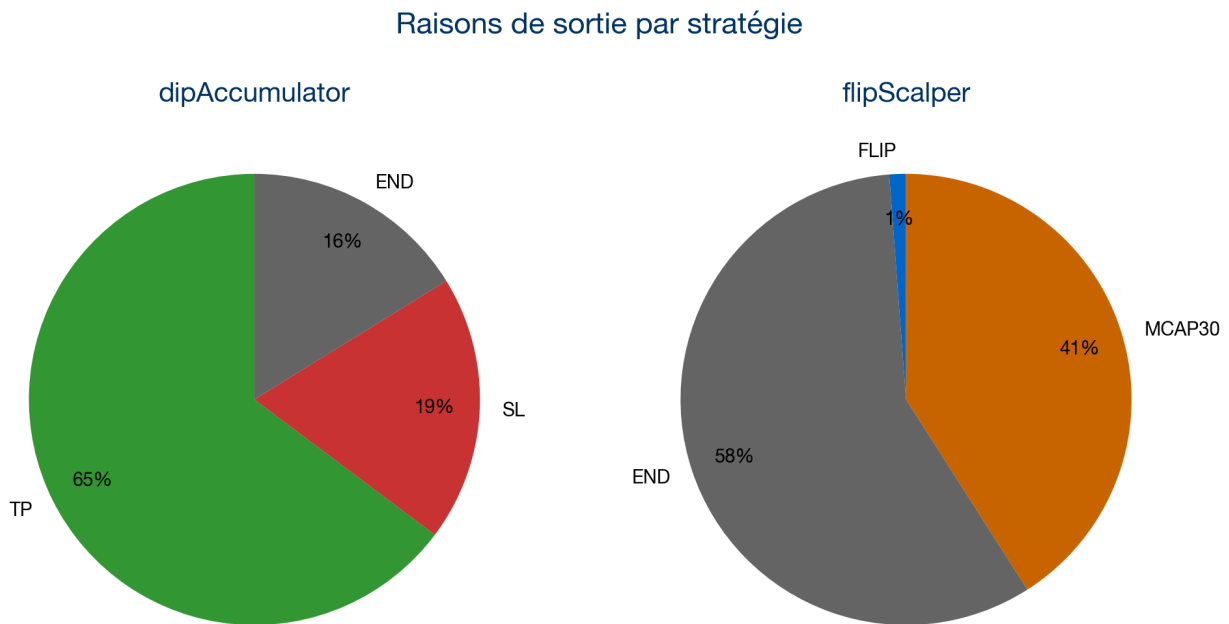


Fig. 12. – Répartition des raisons de sortie par stratégie

7.4 Conclusions

Le bot Mayhem est un **market maker pseudo-aléatoire** dont le smart contract est purement exécutif et le comportement est entièrement piloté off-chain via Jito. Son cycle de vie (activation à $R_s \geq 0.05$ SOL, 50 trades, terminaison par timeout ou erreur Sell ZeroAmount) crée des fenêtres d'opportunité exploitables.

Aucune stratégie n'a permis d'être rentable en toutes circonstances. Le dipAccumulator et le flipScalper affichent des résultats positifs sur certaines runs, mais leur rentabilité dépend fortement des conditions de marché et de la distribution des tokens rencontrés. Une run favorable (majorité de tokens qui pompent) génère des profits, tandis qu'une séquence défavorable peut effacer les gains accumulés. La nature pseudo-aléatoire du bot rend toute garantie de rentabilité impossible.

Les observations clés sont :

1. **Simplicité** : les stratégies simples (dipAccumulator) surperforment les stratégies complexes (smartDipAccumulator, proportionalDip) en conditions live.
2. **Asymétrie** : le TP overshoot favorable (+1.5% en moyenne) compense partiellement le SL overshoot défavorable, mais ne suffit pas à garantir un edge constant.
3. **Biais de validation** : le backtesting offline est nécessaire mais insuffisant, avec un biais de dataset pouvant inverser les conclusions. La validation sur le mainnet est indispensable.